Session 11: Pointers — Notes & Code (C Language)

Mastering Memory Addresses, Dereferencing, Pointer Arithmetic, and Call-by-Reference

October 11, 2026

Contents

1 Topic Notes **2**
- 1.1 What is a Pointer? . . . 2
- 1.2 Key Operators . . . 2
- 1.3 Key Technical Points . . . 2

2 Task 1: Declare Pointer, Print Value and Address **2**
- 2.1 Code . . . 2
- 2.2 Output . . . 3

3 Task 2: Swap Playlist Counts using Pointers **4**
- 3.1 Code . . . 4
- 3.2 Output . . . 4

4 Task 3: Iterate Array using Pointer Arithmetic **5**
- 4.1 Code . . . 5
- 4.2 Output . . . 5

5 Task 4: Increment Follower Counts using Pointer Arithmetic **6**
- 5.1 Code . . . 6
- 5.2 Output . . . 6

6 Task 5: Memory Diagram — Pointer Referencing a Variable **7**
- 6.1 Conceptual Memory Map . . . 7
- 6.2 Component Breakdown . . . 7

7 Quick Revision Summary **7**

1 Topic Notes

1.1 What is a Pointer?

- A pointer is a variable that stores the **memory address** of another variable.
- Declaration: `int *ptr;` → `ptr` is a pointer to an integer.

1.2 Key Operators

Operator	Name	Purpose
`&`	Address-of	Gives the memory address of a variable
`*`	Dereference	Accesses the value stored at the address

Table 1: Pointer Operators in C

1.3 Key Technical Points

- `ptr = &x;` → `ptr` now holds the address of `x`.
- `*ptr` → gives the value stored at that address (dereferencing).
- **Pointer arithmetic:** `ptr + 1` moves the pointer forward by `sizeof(datatype)` bytes (e.g., 4 bytes for `int`).
- Arrays and pointers are closely related — the array name acts like a pointer to its first element.
- Passing pointers to functions allows the function to **modify the original variables** (call by reference).

2 Task 1: Declare Pointer, Print Value and Address

2.1 Code

```c
#include <stdio.h>

int main() {
    int likes = 2500;
    int *ptrLikes;

    ptrLikes = &likes;   // pointer points to likes

    printf("Value of likes        : %d\n", likes);
    printf("Value using pointer   : %d\n", *ptrLikes);
    printf("Address of likes      : %p\n", (void *)&likes);
    printf("Address stored in ptr : %p\n", (void *)ptrLikes);

    return 0;
}
```

Listing 1: Declaring and Using Pointers in C

2.2 Output

```
Value of likes        : 2500
Value using pointer   : 2500
Address of likes      : 0x7ffee4b2c8ac
Address stored in ptr : 0x7ffee4b2c8ac
```

3 Task 2: Swap Playlist Counts using Pointers

3.1 Code

```c
#include <stdio.h>

// Function to swap two integers using pointers
void swapPlaylistCounts(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int playlist1 = 120, playlist2 = 350;

    printf("Before swap:\n");
    printf("Playlist 1: %d songs | Playlist 2: %d songs\n", playlist1,
        playlist2);

    // Pass addresses of the variables
    swapPlaylistCounts(&playlist1, &playlist2);

    printf("\nAfter swap:\n");
    printf("Playlist 1: %d songs | Playlist 2: %d songs\n", playlist1,
        playlist2);

    return 0;
}
```

Listing 2: Swapping Variables Using Pointers (Call by Reference)

3.2 Output

```
Before swap:
Playlist 1: 120 songs | Playlist 2: 350 songs

After swap:
Playlist 1: 350 songs | Playlist 2: 120 songs
```

4 Task 3: Iterate Array using Pointer Arithmetic

4.1 Code

```c
#include <stdio.h>

int main() {
    int orders[5] = {450, 275, 890, 320, 610};
    int *ptr = orders;    // pointer points to first element

    printf("Order Amounts (Zomato):\n");
    for (int i = 0; i < 5; i++) {
        printf("Amount: Rs.%d  |  Address: %p\n", *ptr, (void *)ptr);
        ptr++;    // pointer arithmetic: move to next element
    }

    return 0;
}
```

Listing 3: Iterating Through an Array via Pointer Arithmetic

4.2 Output

```
Order Amounts (Zomato):
Amount: Rs.450  |  Address: 0x7ffee3a1c9a0
Amount: Rs.275  |  Address: 0x7ffee3a1c9a4
Amount: Rs.890  |  Address: 0x7ffee3a1c9a8
Amount: Rs.320  |  Address: 0x7ffee3a1c9ac
Amount: Rs.610  |  Address: 0x7ffee3a1c9b0
```

Note: Each memory address jumps by **4 bytes** (the size of `int`).

5 Task 4: Increment Follower Counts using Pointer Arithmetic

5.1 Code

```c
#include <stdio.h>

void incrementFollowers(int *followers, int n) {
    for (int i = 0; i < n; i++) {
        *followers = *followers + 100;    // modify value at address
        followers++;                      // move to next element
    }
}

int main() {
    int followers[5] = {1500, 2300, 800, 4100, 3200};

    printf("Before increment:\n");
    for (int i = 0; i < 5; i++) {
        printf("Friend %d: %d followers\n", i + 1, followers[i]);
    }

    incrementFollowers(followers, 5);   // array name = address of first
        element

    printf("\nAfter increment (+100 each):\n");
    for (int i = 0; i < 5; i++) {
        printf("Friend %d: %d followers\n", i + 1, followers[i]);
    }

    return 0;
}
```

Listing 4: Modifying Array Elements via Pointer Functions

5.2 Output

```
Before increment:
Friend 1: 1500 followers
Friend 2: 2300 followers
Friend 3: 800 followers
Friend 4: 4100 followers
Friend 5: 3200 followers

After increment (+100 each):
Friend 1: 1600 followers
Friend 2: 2400 followers
Friend 3: 900 followers
Friend 4: 4200 followers
Friend 5: 3300 followers
```

6 Task 5: Memory Diagram — Pointer Referencing a Variable

6.1 Conceptual Memory Map

Using `likes` and `ptrLikes` from Task 1:

```
Memory Address     Variable Name        Value
+----------------+------------------+-----------+
| 0x7ffee4b2c8ac |       likes      |   2500    |   <-- normal int variable
+----------------+------------------+-----------+

                    (ptrLikes stores this address)

+----------------+------------------+-------------------+
| 0x7ffee4b2c8b0 |     ptrLikes     | 0x7ffee4b2c8ac    |   <-- pointer variable
+----------------+------------------+-------------------+

Diagram Flow:

ptrLikes       &likes (address)        likes (value 2500)

   *ptrLikes = 2500   (dereferencing gives the value)
   ptrLikes = 0x7ffee4b2c8ac   (pointer holds the address)
```

6.2 Component Breakdown

- **Variable:** `likes` $\rightarrow$ holds the integer value `2500`.
- **Pointer:** `ptrLikes` $\rightarrow$ holds the memory address `0x7ffee4b2c8ac`.
- **Address:** `0x7ffee4b2c8ac` $\rightarrow$ exact memory location where `likes` resides.

7 Quick Revision Summary

Concept / Operation	Description / Syntax
Address-of Operator	`&` gives address of variable
Dereference Operator	`*` gives value at address
Pointer Initialization	`int *ptr = &var;`
Swapping via Pointers	Modifies original variables directly (call by reference)
Pointer Arithmetic	`ptr++` moves pointer by `sizeof(datatype)` bytes
Array Name Property	`arr == &arr[0]` (acts as address of first element)

Table 2: Pointers Quick Reference

Core Takeaway: Pointers provide direct memory access and enable functions to manipulate caller variables efficiently without copying large blocks of data.